VIET NAM NATIONAL UNIVERSITY HO CHI MINH

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

Final Project

Data Structure Visualizer

Course: Data Structures - CS163

Student list:

Nguyễn Đình Minh Huy (25125083)

xxx (xxxx)

xxx (xxxx)

xxx (xxxx)

Group 09 - Class 25A01

27th April 2026

Contents

1 Abstract **1**

2 Introduction **1**

3 Group Information **1**

4 Data Storage **1**

5 Architecture **2**

5.1 Architectural Overview: MVC Intuition 2

5.2 Detailed System Design and Interfaces 2

5.2.1 The `IVisualizable` Abstraction 3

5.2.2 Timeline and Frames 3

5.3 Visitor Design Pattern for Rendering 4

5.3.1 Pattern Implementation 5

6 Implementation **6**

6.1 Controller Layer Implementation 6

6.1.1 `AppEngine` (Application Orchestrator) 6

6.1.2 `Playback` (Timeline Management) 7

6.2 Model Layer Implementation 7

6.2.1 Singly Linked List 7

6.2.2 Graph Structures (Adjacency List & Adjacency Matrix) 7

6.2.3 Grid Graph (Pathfinding Grids) 8

6.2.4 Heap and AVL Tree 8

6.3 View Layer Implementation 8

7 Technical Problems **9**

8 Features Demo **9**

9 Conclusion **9**

References **10**

1 Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

2 Introduction

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec odio. Praesent libero. Sed cursus ante dapibus diam. Sed nisi. Nulla quis sem at nibh elementum imperdiet.

Duis sagittis ipsum. Praesent mauris. Fusce nec tellus sed augue semper porta. Mauris massa. Vestibulum lacinia arcu eget nulla.

3 Group Information

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus.

Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu.

4 Data Storage

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus volutpat, metus eget egestas mollis, lacus lacus blandit dui, id egestas quam mauris ut lacus.

Fusce vel dui. Pellentesque auctor neque nec urna. Proin sapien ipsum, porta a, auctor quis, euismod ut, mi. Aenean viverra rhoncus pede.

5 Architecture

5.1 Architectural Overview: MVC Intuition

The application strictly adheres to the Model-View-Controller (MVC) design pattern. This architectural choice separates the internal representations of data structures, the logic that manipulates them, and the graphical interfaces presented to the user. This separation of concerns allows for a highly modular, maintainable, and extensible codebase.

The architecture is divided into three interconnected categories of components:

- **Controller**: Orchestrates the application. The central `AppEngine` class serves as the primary coordinator, processing user inputs and coordinating changes across the system. It delegates playback controls to the `Playback` controller, which manages the flow of time for step-by-step animations.

- **Model**: Represents the mathematical data structures, their algorithms, and their visual states over time. The `IVisualizable` interface acts as the base abstraction for all concrete implementations (such as Linked Lists, Graphs, and Heaps). When an algorithm is executed, the structure generates a `Timeline` composed of discrete `Frame`s. Each frame captures the exact state of the structure at a given execution step.

- **View**: Responsible for rendering the visual representation of the `Model`'s state. The `UIManager` handles modern graphical user interface elements via Dear ImGui, providing interactive menus and code panels. The `Renderer` utilizes SFML to draw graphical shapes based on the data contained in the current `Frame`.

5.2 Detailed System Design and Interfaces

A cornerstone of the application's design is its extensive use of abstract interfaces. This approach enforces loose coupling, allowing the front-end rendering logic to remain agnostic to the internal implementations of the back-end data structures.

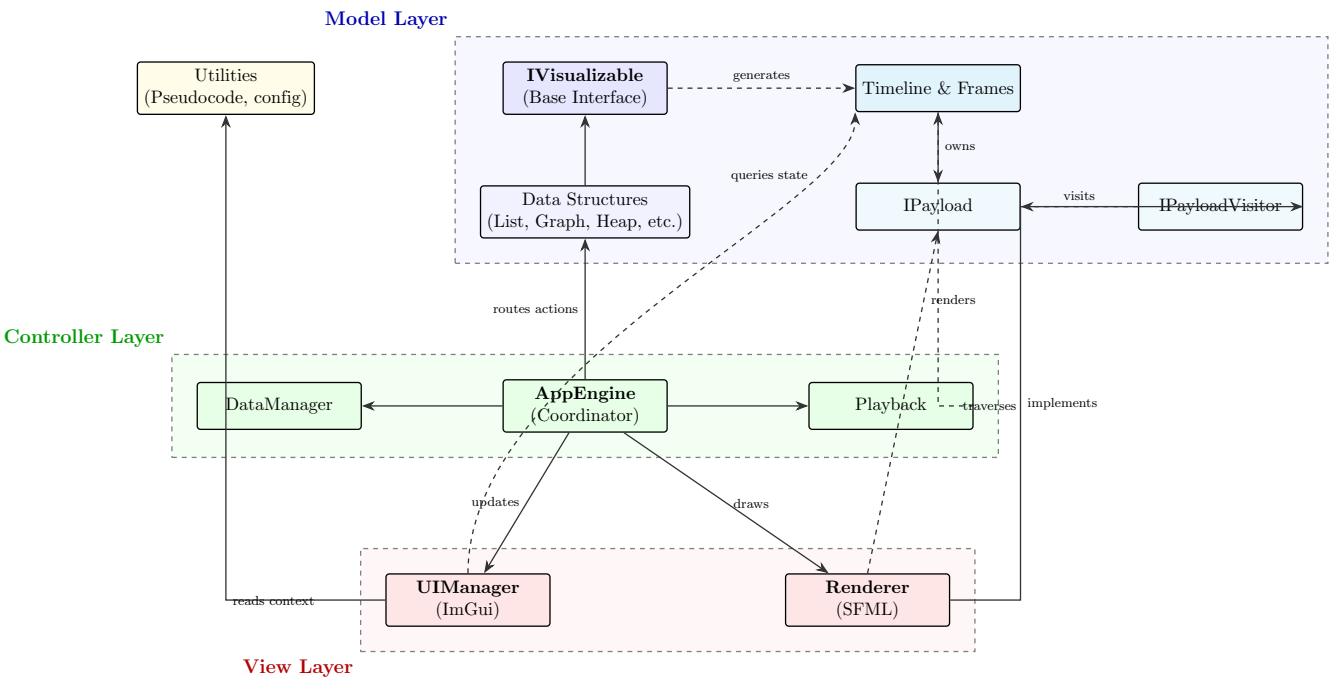

Figure 1: Application Architecture Diagram detailing the relationships between MVC components.

5.2.1 The `IVisualizable` Abstraction

Every data structure in the application inherits from the `IVisualizable` interface. This ensures that the `AppEngine` can interact with an AVL Tree, a Graph, or a Singly Linked List using a unified API. When a user action is dispatched (e.g., inserting a node), the specific data structure executes its algorithm and generates a `Timeline`.

5.2.2 Timeline and Frames

Instead of immediately drawing the results of an algorithm, the models generate discrete states.

- **Timeline**: A collection of `Frame` objects, representing the complete history of an algorithm's execution from start to finish.
- **Frame**: Captures a static snapshot of the visualization at a specific moment in time. A `Frame` holds the highlighted pseudocode line, variable states, and a list of graphical `IPayload` objects that need to be drawn.

This design effectively isolates the algorithmic logic from the real-time rendering loop. It natively supports playback features such as pausing, rewinding, and fast-forwarding, simply by selecting a different `Frame` within the `Timeline`.

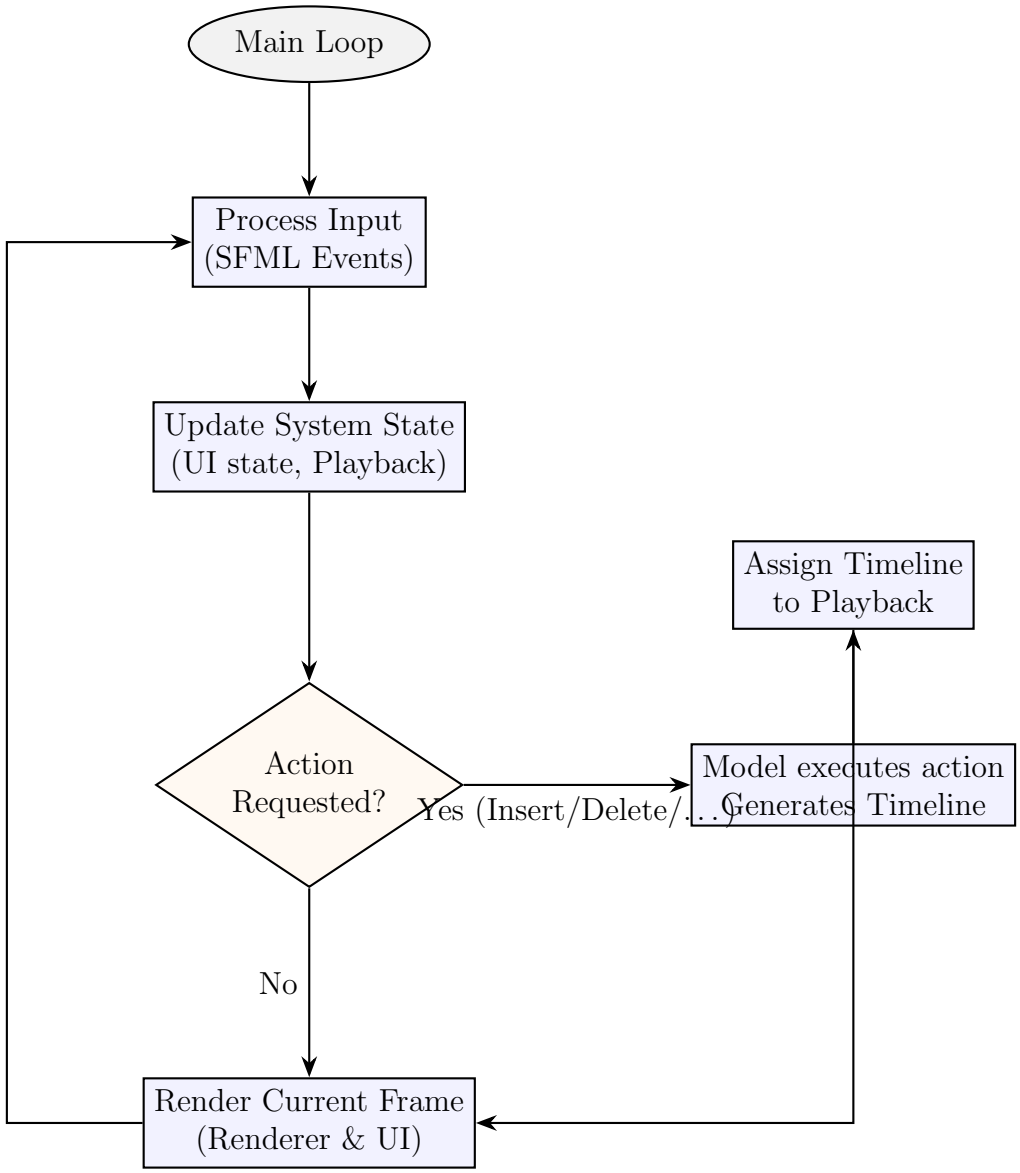

Figure 2: Application Execution Flow Loop.

5.3 Visitor Design Pattern for Rendering

A major challenge in a visualization system is handling the heterogeneous types of data that need to be drawn. A Singly Linked List requires rendering distinct nodes and directional arrows, while an Adjacency Matrix necessitates rendering a grid. If the `Renderer` were to use a long series of `if-else` statements to determine the type of an object before drawing it, the code would quickly become unmaintainable. Furthermore, giving the `Model` the responsibility to render itself would violate the MVC principles.

To solve this problem cleanly, the application employs the **Visitor Design Pattern**.

5.3.1 Pattern Implementation

The pattern relies on two primary abstractions:

1. `IPayloadVisitor`: An interface declaring a `visit()` method for every possible concrete payload type (e.g., `visit(const LinkedListPayload&)`, `visit(const GraphPayload&)`). The `Renderer` implements this interface.

2. `IPayload`: An interface representing a drawing instruction generated by the Model. It requires an `accept(IPayloadVisitor& visitor)` method.

When a `Frame` is rendered, the `Renderer` iterates through the `Frame`'s payload list and calls `payload->accept(*this)`. Through dynamic dispatch, the concrete payload's `accept` method is invoked. This method then immediately calls `visitor.visit(*this)`, effectively passing its strongly-typed self back to the `Renderer`.

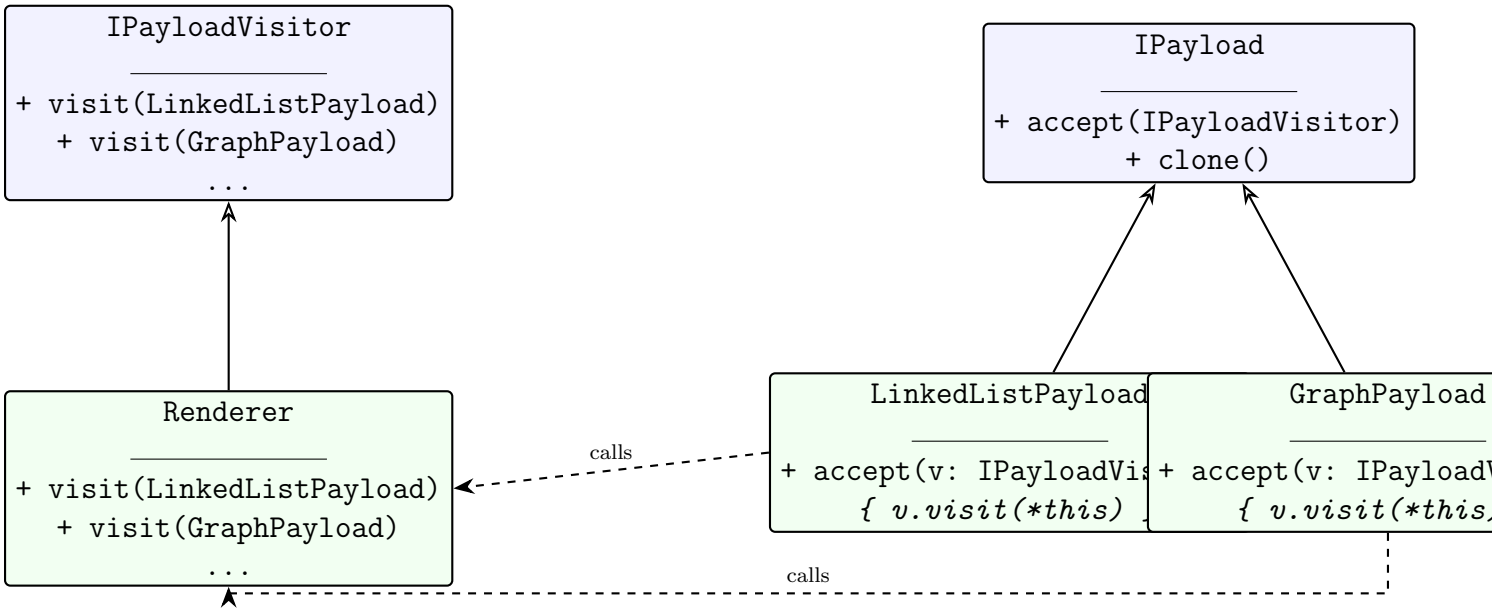

Figure 3: UML Class Diagram illustrating the Visitor Pattern for Payload rendering.

This architecture guarantees that the `Model` never references SFML or rendering specifics, while the `Renderer` avoids costly runtime type checks (`dynamic_cast`), resulting in safe, robust, and highly decoupled code.

6 Implementation

This section details the concrete implementation of the core components within the system architecture. It outlines the specific mechanisms used in the Controller and Model layers, focusing on application orchestration, playback state management, and the underlying data structures utilized for visualizations.

6.1 Controller Layer Implementation

The Controller layer acts as the nervous system of the application, bridging user interactions with underlying structural logic and managing the flow of time.

6.1.1 `AppEngine` (Application Orchestrator)

The `AppEngine` class encapsulates the primary SFML event loop and coordinates all subsystems. Its implementation handles several critical responsibilities:

- **Input Event Processing**: The `AppEngine` captures standard SFML events. If an event is not consumed by the `UIManager` (e.g., Dear ImGui captures a click on a menu button), it falls back to the `Renderer`. This allows seamless interaction, such as dragging and dropping graph nodes directly on the canvas without triggering UI elements.
- **View Manipulation (Zoom and Pan)**: To handle large data structures, `AppEngine` implements an interactive 2D camera system. Mouse scroll wheel events dynamically scale the `sf::View`, adjusting the zoom factor with clamped boundaries to prevent extreme scaling. Panning is implemented by tracking the mouse delta between `MouseButtonPressed` and `MouseMoved` events and shifting the `sf::View` center accordingly.
- **Data Action Routing**: When a user requests an algorithm (e.g., Insert, Delete, Run A*), `AppEngine` queries the `UIManager` for the input parameters. It then resolves the required action to a specific `AlgorithmType`, retrieves the corresponding pseudocode via `PseudocodeManager`, and dispatches the command to the active `IVisualizable` instance. Once the Model generates a `Timeline`, it is immediately assigned to the `Playback` system.

6.1.2 `Playback` (Timeline Management)

The `Playback` system provides VCR-like controls (Play, Pause, Step Forward, Step Backward) over the generated visualization states. Instead of calculating animations on the fly, the `Playback` class acts as an iterator over a fully populated `Timeline`. Its `update(float deltaTime)` method tracks an internal `timeAccumulator`. When the accumulator exceeds the `baseStepDuration` (modified by a `speedMultiplier`), the system advances to the next `Frame` in the timeline. This design guarantees deterministic animation and allows users to jump instantly to any step in the algorithm's execution without recalculating the model's state.

6.2 Model Layer Implementation

The Model layer houses the mathematical logic and the data payload generation mechanisms for the visualizer.

6.2.1 Singly Linked List

The `SinglyLinkedList` class implements the `IStandardStructure` interface. Internally, it manages a standard chain of `Node` pointers.

- **Payload Generation**: Every modification or traversal step in an algorithm (like insertion or searching) records a `Frame`. The structure captures its current state into a `std::vector<int>` and creates a `LinkedListPayload`. It also attaches an array of highlighted indices.
- **Algorithm Tracking**: For instance, during a search operation, the list iterates through its nodes. At each node, a `Frame` is generated with the current node's index highlighted and a descriptive message (e.g., "Comparing with node value X"). This provides the View with the exact context needed to highlight the corresponding visual node on the screen.

6.2.2 Graph Structures (Adjacency List & Adjacency Matrix)

Graphs are implemented using the `IGraphStructure` abstraction, with concrete variants for Adjacency Lists (optimizing sparse graphs) and Adjacency Matrices (optimizing dense graphs).

- **Edge and Vertex Management**: Both variants support adding/removing vertices and directed/undirected edges with weights.

- **Pathfinding Algorithms**: Implementations for algorithms such as Dijkstra, Bellman-Ford, and A* are provided. During execution, the graph maps its internal state (visited nodes, current paths, edge weights) into a generic `GraphPayload`. Nodes and edges that are actively being evaluated are tagged with highlight flags within the payload, allowing the `Renderer` to change their colors dynamically.

6.2.3 Grid Graph (Pathfinding Grids)

The `GridGraph` structure simulates shortest-path algorithms (BFS, A*) on a 2D matrix, treating each cell as a node with implicit edges to its orthogonal and diagonal neighbors.

- **State Mapping**: The grid is represented by an array of `CellState` enums (Empty, Wall, Start, End, Path, Visited).
- **Algorithm Visualization**: When running A*, the algorithm maintains open and closed sets. At each expansion step, a `GridPayload` is generated. Cells in the open set are marked as 'Discovered', and the current shortest path is traced backward from the current node to the start node, marked as 'Path'. This allows users to visually comprehend the expanding frontier of the heuristic search.

6.2.4 Heap and AVL Tree

Note: The detailed implementation of the Heap and AVL Tree structures, including their self-balancing logic and array-mapping visualization mechanics, is handled by other team members and is documented in their respective sections.

6.3 View Layer Implementation

Note: The implementation of the UI components (Dear ImGui integration) and the underlying SFML rendering engine is documented in the subsequent View Layer section by the respective team members.

7 Technical Problems

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor.

Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi.

8 Features Demo

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vivamus luctus urna sed urna ultricies ac tempor dui sagittis. In condimentum facilisis porta.

Sed nec diam eu diam mattis viverra. Nulla fringilla, orci ac euismod semper, magna diam porttitor mauris, quis sollicitudin sapien justo in libero.

9 Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec odio. Praesent libero. Sed cursus ante dapibus diam.

Sed nisi. Nulla quis sem at nibh elementum imperdiet. Duis sagittis ipsum. Praesent mauris.

References

[1] Dan Bernstein. Hash functions (djb2) - comp.lang.c usenet, 1991. Accessed: 2025-12-28.

[2] Eraser.io. Eraser diagram tool. https://eraser.io/, 2025. Online tool used to design UML diagrams and system architecture illustrations.

[3] Google. Gemini. https://gemini.google.com/, 2025. Generative AI model used to support progress tracking, multithreading concepts, cross-platform implementation, code refactoring, and report writing.

[4] OpenAI. Chatgpt. https://chat.openai.com/, 2025. AI assistant used to help draft technical reports, convert content to LaTeX, and format tables, diagrams, and documentation based on student-provided input.